CEU Universidad San Pablo

TÍTULO

Titulo en español

TRABAJO DE FIN DE MÁSTER

EN CIBERSEGURIDAD

Nombre del autor

Julio, 2026

TÍTULO

TÍTULO en inglés

TRABAJO DE FIN DE MÁSTER

EN CIBERSEGURIDAD

Nombre del autor

Nombre del tutor

Julio, 2026

Resumen

Texto con, a lo sumo, 200 palabras.

Palabras Clave

Palabras clave

Abstract

Resumen anterior, en inglés.

Keywords

Keywords

Índice

Capítulo 1 Introducción. 1
1.1 Contexto del TFM 1
1.2 Objetivos 1
1.3 Organización del trabajo 1

Capítulo 2 Gestión del proyecto 2
2.1 Modelo de ciclo de vida 2
2.2 Planificación 2
2.3 Presupuesto (si procede) 2

Capítulo 3 Análisis 3
3.1 Especificación de requisitos 3
3.2 Análisis de los Casos de Uso 4
3.3 Análisis de seguridad 5
3.4 Análisis desde la perspectiva del RGPD (si procede) 5

Capítulo 4 Diseño e implementación 6
4.1 Arquitectura del sistema 6
4.1.1 Arquitectura física 6
4.1.2 Arquitectura lógica 6
4.1.3 Diagrama de infraestructuras de nivel 2 (si procede) 6
4.1.4 Diagrama de infraestructuras de nivel 3 (si procede) 6
4.2 Diseño de datos 7
4.2.1 Migración y carga inicial de datos (si procede) 7
4.3 Diseño de la interfaz de usuario 7
4.4 Diagrama de clases 7
4.5 Entorno de construcción 7
4.6 Referencia al repositorio de software 8

Capítulo 5 Validación del sistema 9
5.1 Plan de pruebas 9
5.2 Evaluación del sistema (si procede) 9

Capítulo 6 Implementación 10
6.1 Conclusiones 10
6.2 Líneas futuras. 10

Índice de figuras

1 Ejemplo ilustrativo de una figura 4

Índice de tablas

1 Ejemplo ilustrativo de una tabla 4
2 Coste estimado del servicio 7

Capítulo 1

Introducción

Todos los capítulos, secciones y apartados de esta plantilla son orientativos. Podrán personalizarse en función del TFM, bajo la supervisión del tutor.

1.1 Contexto del TFM

En este primer capítulo se debe exponer el contexto (temática) del proyecto, justificando por qué surge, en qué consiste, así como la relevancia técnica del mismo.

1.2 Objetivos

Se enumerarán los objetivos principales del proyecto. Preferiblemente, identificando claramente el objetivo principal, así como los diferentes subobjetivos planteados.

1.3 Organización del trabajo

Se indicará cómo se estructura el documento (contenido de cada capítulo).

Capítulo 2

Gestión del proyecto

2.1 Modelo de ciclo de vida

Se deberá indicar la metodología de desarrollo seguida, es decir, si se ha llevado a cabo un modelo de ciclo de vida en cascada, incremental, etc.

2.2 Planificación

Se identificarán las tareas a realizar, su descripción, sus fechas estimadas de inicio y de fin, sus precedencias, los recursos necesarios, estimación de esfuerzo, etc. Se podrán utilizar diagramas de Gantt, PERT / CPM, etc. Deberán incluirse (si procede) también los diferentes hitos a lo largo del proyecto. Para que el estudiante tenga la experiencia real de planificar y controlar su cumplimiento, es **importante** que la planificación se realice con anterioridad a la finalización del proyecto.

Se pueden encontrar explicaciones tanto sobre los diagramas que se indican en esta sección como los que se mencionan en los siguientes apartados en la documentación sobre técnicas de la metodología Métrica 3.

2.3 Presupuesto (si procede)

En caso de que el desarrollo del TFG en sí mismo implique costes, o tenga sentido un trabajo con proyección a futuro, se indicará qué personal (con sus perfiles, dedicación, etc.), licencias de software, infraestructuras, etc. serán necesarios, así como sus precios y costes correspondientes.

Capítulo 3

Análisis

3.1 Especificación de requisitos

En un escenario habitual, se deberán especificar, al menos, los siguientes tipos de requisitos:

- Funcionalidades del sistema, que podrán detallarse, por ejemplo, mediante diagramas de casos de uso con su especificación. Alternativamente se pueden mostrar historias de usuario.
- Interfaz de usuario (si procede). Puede incluirse, por ejemplo, el nivel de cumplimiento de la norma de accesibilidad WCAG.
- Rendimiento (si procede), incluyendo tiempos de respuesta.
- Capacidad (si procede), incluyendo el número máximo de peticiones que se pueden procesar por unidad de tiempo.
- Seguridad (si procede). Se indicará el nivel de seguridad que ha de satisfacer el sistema, por ejemplo, según el Esquema Nacional de Seguridad (RD 3/2010) o según alguna norma de la familia ISO 27000. Este nivel de seguridad se establecerá a partir de un análisis de riesgos (véase la sección de análisis de seguridad).
- Interoperabilidad con otros sistemas (si procede).
- Protección de datos (si procede), en concreto, el cumplimiento del Reglamento General de Protección de Datos ([UE] 2016/679) y de la Ley Orgánica 3/2018 de Protección de Datos Personales y Garantía de los Derechos Digitales (véase la sección de análisis desde la perspectiva del RGPD).
- Requisitos sobre entorno tecnológico y de comunicaciones (si procede), es decir,

requisitos sobre gestores de bases de datos, sistema o los sistemas operativos, hardware, protocolos de red, etc.

Tabla 1 Ejemplo ilustrativo de una tabla

Ref.	**Descripción**
RF01	Descripción del requisito funcional 01.
RF02	Descripción del requisito funcional 02.
…	…
RFN	Descripción del requisito funcional N.

3.2 Análisis de los Casos de Uso

Deberán identificarse los diferentes casos de uso del sistema. Podrá elaborarse un Diagrama de Casos de Uso y/o Diagrama de Contexto del sistema. Si es necesario elaborar varios diagramas de casos de uso, en esta sección se incluirán sólo los más relevantes. El resto se podrán incluir en un anexo.

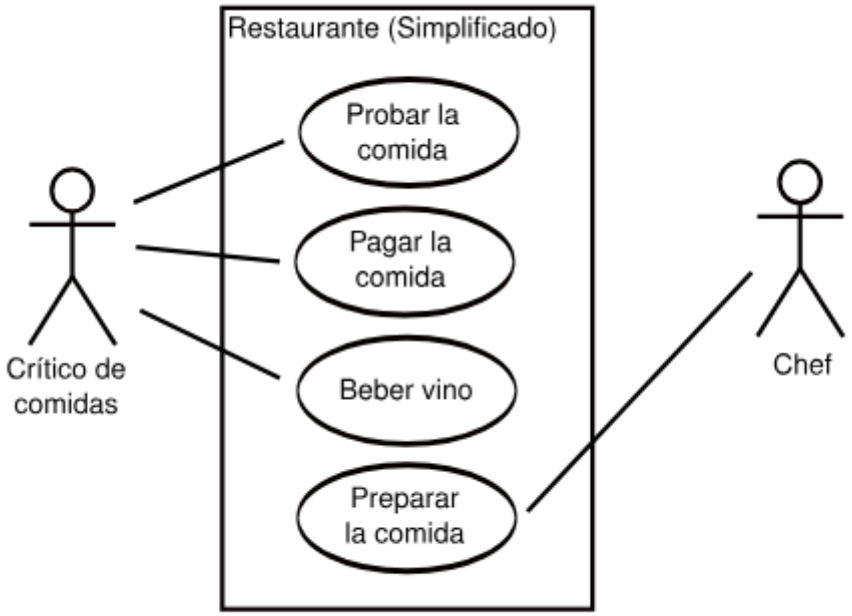

Figura 1 Ejemplo ilustrativo de una figura

Los diagramas elaborados no estarán condicionados por la tecnología utilizada, sino que estarán centrados en el problema en sí a resolver.

3.3 Análisis de seguridad

Se llevará a cabo un análisis de riesgos de acuerdo con las dimensiones de autenticidad, confidencialidad, integridad, disponibilidad y trazabilidad. Una página a lo sumo. Si se necesitara más, se incluirían en un anexo.

3.4 Análisis desde la perspectiva del RGPD (si procede)

En caso de que sea necesario, se llevará a cabo una gestión del riesgo y evaluación de impacto en tratamientos de datos personales.

Capítulo 4

Diseño e implementación

4.1 Arquitectura del sistema

Se elaborarán diagramas donde se indiquen las interfaces del sistema, los diferentes servicios que proporciona, los componentes en que se apoyan tales servicios, capa de acceso a datos, etc. Los diagramas de arquitectura NO deberían estar ligados a ninguna tecnología, sistema o implementación concreta.

4.1.1 Arquitectura física

Mostrará todos los componentes físicos que intervienen en un sistema para su despliegue y funcionamiento (servidores, conexiones, clientes…).

4.1.2 Arquitectura lógica

Si se trata de una arquitectura en tres capas, los diferentes subsistemas y módulos. Si se trata de una arquitectura basada en microservicios, cada microservicio, gestión de comunicaciones, gateway, etc.

4.1.3 Diagrama de infraestructuras de nivel 2 (si procede)

Se mostrarán los routers, switches, conexiones entre ellos, etc. Para cada uno de los dispositivos se indicarán sus características técnicas.

4.1.4 Diagrama de infraestructuras de nivel 3 (si procede)

Se mostrarán las diferentes zonas de seguridad, firewalls, VLANs, servidores, etc. Los componentes de la infraestructura podrán ser tanto materiales como virtuales.

Tabla 2 Coste estimado del servicio

Ref.	**Descripción**
RF01	Descripción del requisito funcional 01.
RF02	Descripción del requisito funcional 02.
…	…
RFN	Descripción del requisito funcional N.

4.2 Diseño de datos

Diagrama Entidad-Relación extendido o equivalente. Se llevará un proceso de normalización y de optimización para obtener las tablas (modelo físico) de la base de datos.

4.2.1 Migración y carga inicial de datos (si procede)

En caso de que sea necesario, se elaborará un plan de migración y carga inicial de datos.

4.3 Diseño de la interfaz de usuario

Se mostrará cómo serán las ventanas, la navegación, etc. En caso de que sea necesario, podrán utilizarse diagramas de transición de estados.

4.4 Diagrama de clases

A lo largo de este documento se ha asumido un desarrollo orientado a objetos. No obstante, se pueden seguir otros enfoques, por ejemplo, un desarrollo estructurado con diagrama de contexto, diagramas de flujos de datos, diagrama entidad-relación, etc. Por otra parte, el tutor del proyecto puede dar sus propias orientaciones si lo considera oportuno.

4.5 Entorno de construcción

Se indicará el IDE, frameworks, simuladores, librerías, etc. que se han utilizado, así como otros detalles relevantes para la implementación del proyecto.

4.6 Referencia al repositorio de software

Se indicará la URL del repositorio en el que está el software para, si procede, que el tribunal pueda probarlo en sus propias máquinas. Si la aplicación puede ser desplegada en un sitio públicamente accesible, se deberá proporcionar la URL del sitio, así como toda la información necesaria para acceder al sistema.

Si este capítulo resultase demasiado extenso, podría separarse en dos.

Capítulo 5

Validación del sistema

5.1 Plan de pruebas

Se deberá desarrollar un Plan de Pruebas del sistema: pruebas unitarias, de integración, y/o End-to-End. Se establecerán los criterios para codificar las pruebas realizadas, por ejemplo, a través de Test Driven Development (TDD). Asimismo, se diseñarán el resto de las pruebas que sean necesarias: estrés, carga, evaluación de la accesibilidad, a través de scripting (p.ej., con Selenium), con usuarios reales, etc.

5.2 Evaluación del sistema (si procede)

Métricas aplicadas para la evaluación del sistema (funcionamiento, usuario, etc.).

Capítulo 6

Implementación

Ideas a las que se llega después del desarrollo del proyecto, así como las líneas posibles de trabajo posterior.

6.1 Conclusiones

fsdfsdfsdf APIs dfsdfsdf sistemas distribuidos[1]

gfdgadfgadffgadf **android_developers_architectures**

Conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones conclusiones

6.2 Líneas futuras

Líneas futuras líneas futuras

[1] **uvigo_cliente_servidor**, p. 1

Bibliografía

[1] I. Somerville, *Software Engineering*, 10.ª ed. Boston: Pearson Education, 2015.

[2] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Tesis doct., University of California, 2000.

[3] Iniciativa Aporta. "Buenas prácticas en el diseño de APIs y linked data." Gobierno de España. dirección: https://datos.gob.es/elearning/Unidades_Didacticas/Unidad_8/contenidos/des

[4] Oracle. "MySQL 8.0 Reference Manual," visitado ene. de 2026. dirección: https://dev.mysql.com/doc/

[5] J. Gu, M. Goetschalckx y L. McGinnis, "Research on warehouse design and performance evaluation: A comprehensive review," *European Journal of Operational Research*, vol. 203, n.º 3, págs. 539-549, 2010.

Anexo I – Manuales

En caso de que no tenga sentido incluir en el fichero README.md del repositorio alguno de los siguientes manuales: instalación, operación o usuario, se incluirá en la memoria del TFM.

A lo largo de este documento se ha asumido un desarrollo orientado a objetos. No obstante, se pueden seguir otros enfoques, por ejemplo, un desarrollo estructurado con diagrama de contexto, diagramas de flujos de datos, diagrama entidad-relación, etc. Por otra parte, el tutor del proyecto puede dar sus propias orientaciones si lo considera oportuno.

Anexo II - Ejemplos de citas, bibliografía y glosario

Este anexo recoge ejemplos de uso de los comandos personalizados definidos en la plantilla para comprobar cómo se muestran en el PDF final las citas, las referencias bibliográficas, las siglas y las definiciones del glosario.

Ejemplos de siglas y términos del glosario

En este párrafo se fuerzan varias apariciones de entradas del glosario y siglas para comprobar su comportamiento. Un Backend puede construirse sobre un Framework moderno y exponer una API de tipo REST. En algunos contextos, un ERP puede integrarse con un WMS para compartir información logística. Además, en una implementación Java, el uso de JDBC permite obtener un ResultSet como resultado de una consulta a base de datos.

También se puede comprobar la diferencia entre formas corta y larga de las siglas. Por ejemplo, Interfaz de Programación de Aplicaciones es el desarrollo completo de API, mientras que Transferencia de Estado Representacional desarrolla la sigla REST. De forma similar, Planificación de Recursos Empresariales y Sistema de Gestión de Almacenes muestran el nombre extendido de cada término.

Ejemplos de citas en nota al pie

A continuación se muestran ejemplos de todos los comandos de cita definidos en la plantilla.

Ejemplo de cita textual con página usando \footcitepage: "Software engineering is concerned with theories, methods and tools for professional software development"[2].

Ejemplo de nota al pie con referencia y página usando \footbibliographypage. Este comentario incluye únicamente la referencia en nota al pie[3] para comprobar su aspecto visual.

[2] I. Somerville, *Software Engineering*, 10.ª ed. Boston: Pearson Education, 2015, p. 7

[3] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Tesis

Ejemplo de cita web completa usando \footweb. La guía sobre APIs y linked data puede consultarse en la referencia correspondiente[4].

Ejemplo de cita web con fragmento usando \footwebfragment. En este caso se añade un fragmento relevante de la fuente web[5].

Ejemplo de cita web con texto citado usando \footwebcite: "Buenas prácticas en el diseño de APIs y linked data"[6].

Ejemplo de definición con referencia usando \footcitedefinition: La ingeniería del software puede entenderse como una disciplina orientada al desarrollo profesional de sistemas software[7].

Ejemplo combinado

El estudio de un WMS puede apoyarse en literatura académica especializada. Por ejemplo, "Research on warehouse design and performance evaluation"[8] permite comprobar el estilo de cita textual con página. Del mismo modo, una arquitectura REST puede justificarse mediante trabajos académicos previos[9], mientras que la documentación técnica de plataformas concretas puede citarse como fuente web[10].

doct., University of California, 2000, p. 76

[4]Iniciativa Aporta. "Buenas prácticas en el diseño de APIs y linked data." Gobierno de España. dirección: https : //datos.gob.es/elearning/Unidades_Didacticas/Unidad_8/contenidos/descargas/unidad_imprimible.pdf

[5]Oracle. "MySQL 8.0 Reference Manual," visitado ene. de 2026. dirección: https : //dev.mysql.com/doc/ Fragmento: The MySQL Reference Manual contains detailed information about server administration, SQL syntax and APIs

[6]Iniciativa Aporta. "Buenas prácticas en el diseño de APIs y linked data." Gobierno de España. dirección: https : //datos.gob.es/elearning/Unidades_Didacticas/Unidad_8/contenidos/descargas/unidad_imprimible.pdf Fragmento: Documento institucional del Gobierno de España sobre diseño de APIs y linked data

[7]I. Somerville, *Software Engineering*, 10.ª ed. Boston: Pearson Education, 2015, p. 5

[8]J. Gu et al., "Research on warehouse design and performance evaluation: A comprehensive review," *European Journal of Operational Research*, vol. 203, n.º 3, págs. 539-549, 2010, p. 539

[9]R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Tesis doct., University of California, 2000, p. 50

[10]Oracle. "MySQL 8.0 Reference Manual," visitado ene. de 2026. dirección: https : //dev.mysql.com/doc/

Un sistema real suele componerse de un Backend, uno o varios servicios expuestos mediante API y mecanismos de persistencia gestionados con tecnologías como JDBC. Todo ello puede implementarse con ayuda de un Framework, cuya definición también aparecerá posteriormente en el glosario.

Texto de relleno para comprobar múltiples referencias

Este párrafo no busca coherencia temática, sino mostrar varias llamadas seguidas a comandos personalizados. La expresión "professional software development"[11] permite ver cómo se integra una cita en línea con el resto del texto. A continuación se añade una referencia bibliográfica aislada[12]. Después se añade una referencia web con fragmento[13]. Finalmente, se incluye otra cita textual web para comprobar el espaciado y el formato"Reference Manual"[14].

[11] I. Somerville, *Software Engineering*, 10.ª ed. Boston: Pearson Education, 2015, p. 3

[12] J. Gu et al., "Research on warehouse design and performance evaluation: A comprehensive review," *European Journal of Operational Research*, vol. 203, n.º 3, págs. 539-549, 2010, p. 541

[13] Iniciativa Aporta. "Buenas prácticas en el diseño de APIs y linked data." Gobierno de España. dirección: https : //datos.gob.es/elearning/Unidades_Didacticas/Unidad_8/contenidos/descargas/unidad_imprimible.pdf Fragmento: Se describen recomendaciones sobre diseño e interoperabilidad de APIs

[14] Oracle. "MySQL 8.0 Reference Manual," visitado ene. de 2026. dirección: https : //dev.mysql.com/doc/ Fragmento: Información oficial sobre administración, sintaxis SQL y conectores

Glosario de siglas

API Interfaz de Programación de Aplicaciones.

ERP Planificación de Recursos Empresariales.

REST Transferencia de Estado Representacional.

WMS Sistema de Gestión de Almacenes.

Definiciones

Backend Conjunto de servidores y componentes software encargados de proporcionar servicios, gestionar la lógica de negocio y permitir el acceso a recursos y datos desde aplicaciones cliente.

Framework Estructura de desarrollo compuesta por componentes de software reutilizables, librerías y convenciones que facilitan la creación de aplicaciones de forma estandarizada y eficiente.

JDBC Java Database Connectivity, API estándar de Java que permite a las aplicaciones acceder y gestionar bases de datos relacionales mediante la ejecución de consultas SQL y la manipulación de los resultados obtenidos.

ResultSet Estructura de datos utilizada en JDBC para representar el conjunto de resultados devuelto por una consulta a una base de datos, permitiendo recorrer fila a fila los registros obtenidos y acceder a los valores de cada columna.